

多媒体第五次实验

1. 完成本次实验任务的思路

本次实验的主要任务是利用 Python 实现霍夫曼编码算法，并完成对文本文件的压缩处理。霍夫曼编码是一种经典的无损数据压缩算法，其核心思想是根据字符在文本中出现频率的不同，为高频字符分配较短的二进制编码，为低频字符分配较长的编码，从而降低整体编码长度，提高存储效率。

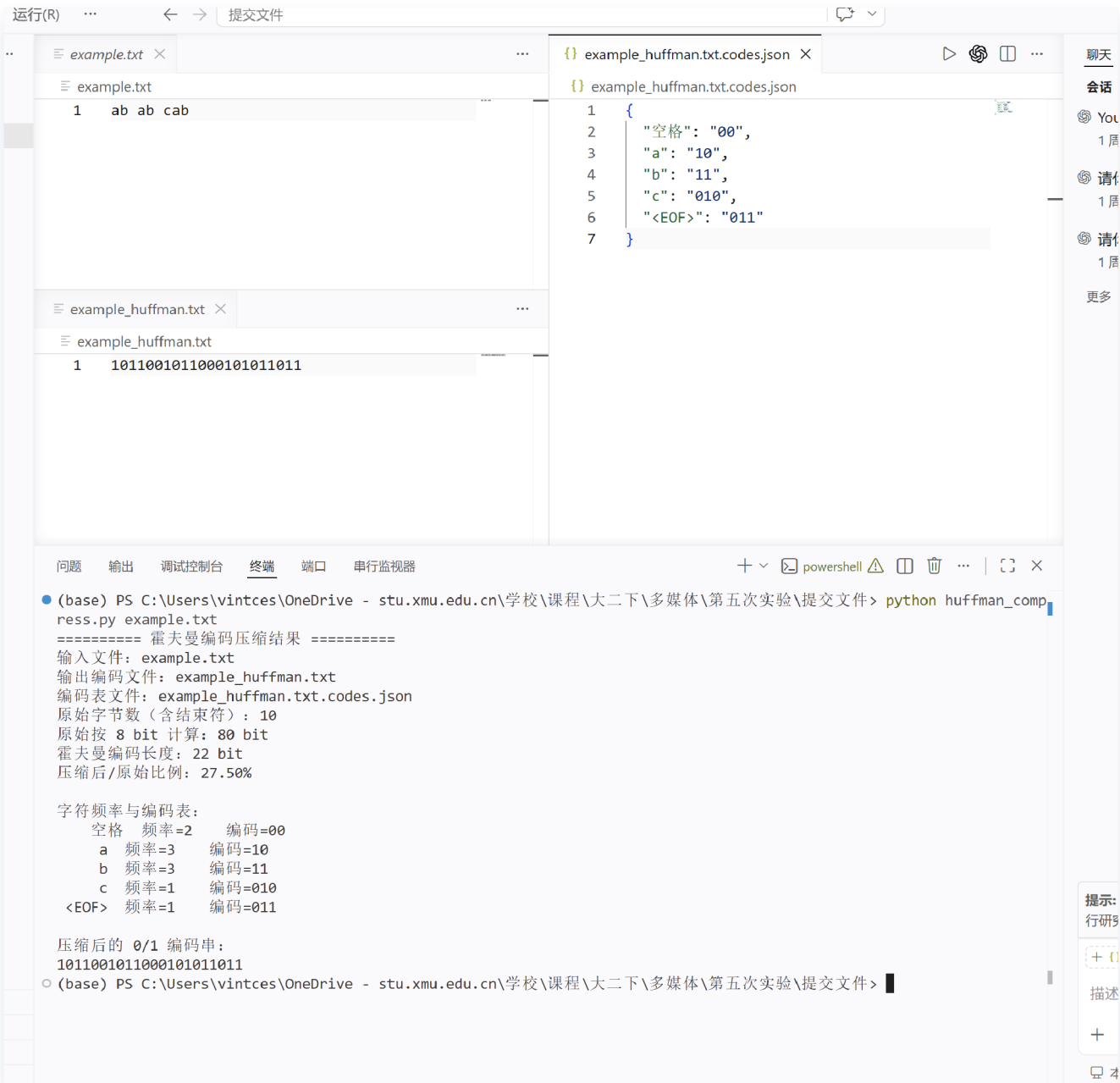
实验开始时，首先需要读取输入文本文件中的全部内容，并统计每个字符在文本中出现的频率。为了保证编码过程完整，还额外加入结束符 EOF，用于表示文件结束。随后根据字符频率构建霍夫曼树。构建过程中，持续从优先队列中取出频率最小的两个节点，将其合并为新的父节点，再重新加入队列，直到最终形成唯一根节点。之后通过递归遍历霍夫曼树，为每个字符生成对应的霍夫曼编码，其中左子树记为 0，右子树记为 1。

得到编码表后，程序会将原始文本逐字符转换为对应的二进制字符串，并输出压缩后的霍夫曼编码结果。同时程序还会将字符编码表保存为 JSON 文件，便于后续解码与分析。为了验证霍夫曼编码的实际压缩效果，本实验分别对简单短文本和较长文本进行了测试，并统计原始文本大小、压缩后编码长度以及压缩率，从而分析霍夫曼编码对不同规模文本的压缩性能。

整个程序采用 Python 编写，主要利用 `heapq` 优先队列实现霍夫曼树构建，利用 `collections.Counter` 统计字符频率，并结合 JSON 文件实现编码表输出。程序整体结构清晰，能够自动完成文本读取、编码生成、压缩输出以及结果统计等功能，实现了完整的霍夫曼编码文本压缩流程。

2. 运行程序截图和简要说明

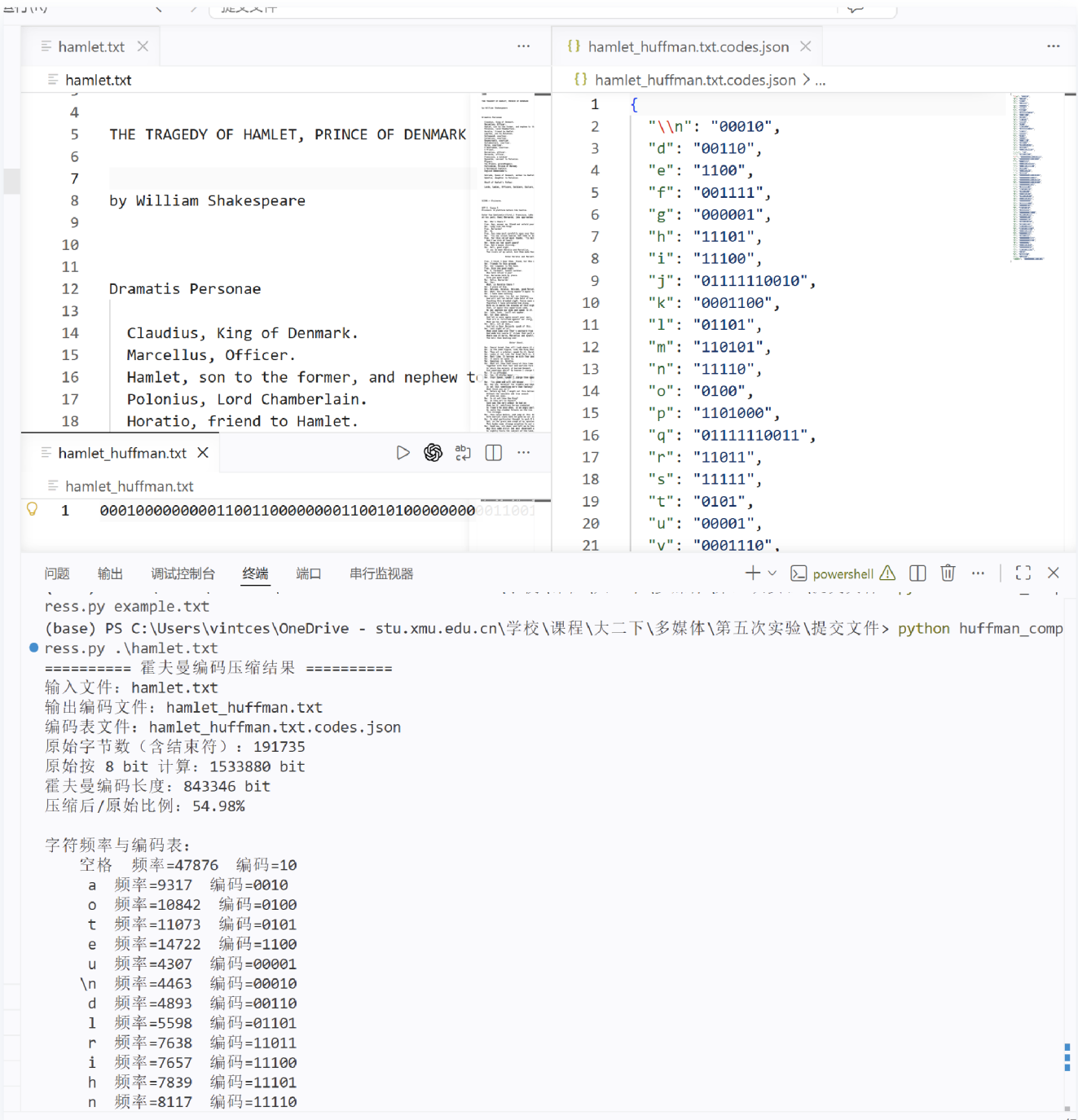
(1) example.txt 原始文件、压缩结果、编码表与程序运行结果截图



简要说明：

该截图展示了 example.txt 原始文本内容、生成的霍夫曼编码结果、字符编码表以及程序运行控制台输出。程序能够正确统计字符频率并生成对应霍夫曼编码，同时输出压缩后的二进制字符串。控制台中还显示了原始文件大小、压缩后编码长度以及压缩率等信息，验证了霍夫曼编码算法的正确性。

(2) hamlet.txt 长文本压缩结果与编码表截图



简要说明:

该截图展示了长文本文件 hamlet.txt 的压缩实验结果。程序能够正确处理较大规模文本，并生成对应的霍夫曼编码文件与编码表。控制台输出中显示压缩后数据长度明显减少，说明霍夫曼编码对于高频字符较多的大规模文本具有较好的压缩效果。同时编码表中可以观察到空格、字母 e、字母 t 等高频字符获得了较短编码，符合霍夫曼编码“高频短码、低频长码”的设计思想。

3. 核心代码展示和分析

(1) 统计字符频率

```
from collections import Counter

freq = Counter(text)
freq["<EOF>"] += 1
```

代码分析：

该部分代码用于统计文本中每个字符出现的频率。程序利用 Python 的 Counter 类快速完成字符计数，并额外加入 EOF 结束符，用于标识文本结束。字符频率是构建霍夫曼树的重要依据，字符出现次数越高，其最终生成的编码长度越短。

(2) 构建霍夫曼树

```
while len(heap) > 1:
    left = heapq.heappop(heap)
    right = heapq.heappop(heap)

    merged = Node(
        left.freq + right.freq,
        None,
        left,
        right
    )

    heapq.heappush(heap, merged)
```

代码分析：

该部分代码用于构建霍夫曼树。程序不断从优先队列中取出频率最小的两个节点，并将它们合并为新的父节点，再重新加入队列。由于每次优先合并最小频率节点，因此最终能够保证整体编码长度最短，从而达到最佳压缩效果。

(3) 生成霍夫曼编码

```
def generate_codes(node, prefix="", codebook=None):
    if codebook is None:
        codebook = {}

    if node.char is not None:
        codebook[node.char] = prefix
        return codebook

    generate_codes(node.left, prefix + "0", codebook)
    generate_codes(node.right, prefix + "1", codebook)

    return codebook
```

代码分析：

该函数通过递归遍历霍夫曼树生成对应编码。遍历左子树时追加字符 0，遍历右子树时追加字符 1。当遍历到叶子节点时，即可得到对应字符的霍夫曼编码。由于霍夫曼树不存在前缀冲突，因此生成的编码能够唯一解码。

(4) 文本压缩编码

```
encoded = "".join(codes[ch] for ch in text)
encoded += codes["<EOF>"]
```

代码分析：

该部分代码将原始文本逐字符替换为对应的霍夫曼编码，并拼接形成最终压缩后的二进制字符串。最后额外加入 EOF 结束符编码，确保解码时能够正确判断文件结束位置。

(5) 保存编码表

```
with open(codebook_file, "w", encoding="utf-8") as f:
    json.dump(codes, f, ensure_ascii=False, indent=4)
```

代码分析：

程序利用 JSON 格式保存字符编码表，便于后续查看与解码处理。由于设置了 `ensure_ascii=False`，因此能够正确保存中文与特殊字符，提高程序通用性。

(6) 压缩率统计

```
original_bits = len(text) * 8
compressed_bits = len(encoded)

ratio = compressed_bits / original_bits * 100
```

代码分析：

程序将原始文本按每字符 8 bit 进行统计，并与霍夫曼编码后的总 bit 数进行比较，从而计算压缩率。实验结果表明，文本规模越大、字符重复率越高，霍夫曼编码压缩效果越明显。

(7) 实验结果分析

通过本次实验可以发现，霍夫曼编码能够有效减少文本存储空间。对于 `example.txt` 等简单文本，程序已经能够正确完成编码与压缩；而对于 `hamlet.txt` 等大规模文本，压缩效果更加明显。实验中高频字符获得较短编码，而低频字符采用较长编码，这正体现了霍夫曼编码根据字符频率动态优化编码长度的核心思想。

本实验不仅加深了对霍夫曼编码原理的理解，也进一步熟悉了 Python 文件处理、优先队列、递归算法以及 JSON 数据处理等相关知识，为后续多媒体数据压缩学习奠定了基础。